

Interview Question Bank: Tech Lead (Front-End / UI Architecture)

Instructions: Use this as a pick-list. Start with role-fit and recent work, then go deep on 1–2 projects, and finish with system design and leadership scenarios. Ask for examples, trade-offs, and measurable outcomes.

60-Min Tech Lead Interview Plan

Time	Focus	What to Ask (Angular-heavy)
0–5 min	Set context	Confirm Angular depth (versions used, last project in Angular). Clarify role scope: owning front-end architecture, standards, mentoring, and delivery.
5–12 min	Resume deep-dive (pick an Angular project)	Personal ownership, hardest UI problem, measurable outcomes. Probe Tech Lead actions: sprint risk mgmt, review practices, tech debt decisions.
12–27 min	Angular architecture & system design	Design a scalable Angular app for a data-heavy dashboard. Cover: module boundaries, routing strategy (lazy loading), shared libs, state strategy, API contracts, error handling, observability.
27–42 min	Angular technical depth (hands-on)	Use Section 3 questions. Prioritize: RxJS patterns, change detection, performance pitfalls, reactive forms, interceptors/guards/resolvers, upgrade strategy.
42–52 min	Testing, quality & build/release	Unit testing with Jasmine/Karma (what to mock, Observables, async). Approach for e2e. CI standards (linting, type checks), release strategy, preventing regressions.
52–58 min	Leadership & delivery	Mentoring approach, setting coding standards, reviewing RxJS/NgRx usage, handling conflicting priorities, and managing technical debt.
58–60 min	Close	Candidate questions + sell the role. Align on next steps and any gaps to validate.

Angular-Heavy Focus Checklist (What “Strong” Looks Like)

- **Architecture:** Feature modules + lazy routes; shared UI library strategy; clear boundaries for core/shared/feature code.
- **RxJS:** Understands cancellation, error handling, multicasting, and avoiding nested subscriptions.

- **State:** Can justify NgRx (or alternatives) and design effects/selectors without over-engineering.
- **Performance:** Knows change detection mechanics; uses OnPush/trackBy; can profile and fix slow views.
- **Forms:** Reactive forms, dynamic forms, complex validation, and clean form state management.
- **Testing:** Reliable unit tests for services/components/streams; pragmatic integration/e2e strategy.
- **Upgrades:** Structured approach for Angular version upgrades with de-risking and CI gates.

Scorecard (Quick Rubric)

- **Technical depth (Angular):** RxJS fluency, change detection/performance, routing/modules, forms, DI, and testing approach.
- **Architecture:** Scalable module design, shared libraries, state strategy, and upgrade/versioning plan that keeps multiple apps stable.
- **Delivery & quality:** Pragmatic, test strategy, CI/code quality enforcement, handles incidents and reliability.
- **Leadership:** Mentoring, delegation, conflict handling, stakeholder communication, tech debt ownership.
- **Communication:** Structured explanations, asks clarifying questions, aligns solutions to business outcomes.

Suggested Follow-Ups (Signals)

- **Green flags:** Gives concrete examples with metrics; explains trade-offs; discusses failure modes; shows ownership (standards, reviews, de-risking upgrades); demonstrates empathy for users (a11y, UX, reliability).
- **Red flags:** Only lists tools without decisions; vague on personal contribution; can't explain state/performance/testing; blames stakeholders/teams; no strategy for maintainability or incident handling.

1) Role-Fit & Overview

- Walk me through your last 2–3 years of work. What were you personally accountable for?
- You mention 14+ years in UI/Front-End architecture. What does “architecture” mean in your day-to-day role?
- What kind of products have you built (B2B/B2C/internal tools)? Which had the highest scale or complexity?
- Which framework do you consider your strongest today (Angular/React/Vue) and why?
- Describe a decision you made that improved delivery speed or quality across the team.

2) UI Architecture & Front-End System Design

- How do you structure a large front-end codebase (monorepo vs multirepo, feature-based folders, shared packages)?

- When do you choose micro-frontends? What integration patterns have you used (module federation, iframe, custom elements)?
- How do you design reusable components so they remain flexible without becoming “god components”?
- Explain your approach to state management at different scales (local state, context, Redux/NgRx, server state).
- How do you handle configuration-driven UIs (feature flags, dynamic forms, tenant-based config)?
- What are your standards for accessibility and design systems? How do you enforce them?
- How do you version and publish shared UI libraries, and prevent breaking changes across apps?

3) Angular (1.x to 12+) Deep Dive

- What were the biggest differences you faced moving from AngularJS (1.x) to Angular (2+)?
- Describe an upgrade you led (e.g., Angular 9 → 12). What broke, and how did you de-risk it?
- How do you manage performance in Angular (OnPush, trackBy, lazy loading, change detection pitfalls)?
- How do you structure modules and routing for a large enterprise app?
- Explain interceptors/guards/resolvers—where have you used each and why?
- What’s your approach to forms (template-driven vs reactive), dynamic forms, and validation?
- How have you used RxJS in production? Explain higher-order mapping operators and cancellation (switchMap vs mergeMap vs concatMap).
- Testing: how do you write unit tests for components/services? What do you mock and what do you integration-test?

4) ReactJS Deep Dive

- Describe how you design a React component API (props, composition, render props/hooks) for reuse.
- How do you manage state and side-effects (useState/useReducer, Context, Redux, React Query)?
- What are common causes of unnecessary re-renders? How do you detect and fix them?
- Explain hooks rules and a real bug you’ve seen due to dependencies in useEffect/useMemo/useCallback.
- How do you handle routing, code splitting, and lazy loading in React apps?
- Testing approach (Jest/RTL): what do you unit test vs integration test?
- If you migrated a feature from Angular/Vue to React, what were the key differences in patterns?

5) Vue.js / Nuxt.js

- Where does Vue work best for you compared to Angular/React?
- Nuxt: how do you decide between SSR, SSG, and SPA mode? What trade-offs did you face?
- How do you manage state in Vue (Pinia/Vuex), and what patterns keep it maintainable?

- How do you handle component communication (props/events, provide/inject) and avoid tight coupling?
- Vuetify/Tailwind: how do you balance speed of delivery with consistent UX and performance?

6) JavaScript (ES6+) & TypeScript Fundamentals

- Explain event loop, microtasks vs macrotasks, and how Promises/async-await fit in.
- What are closures and where have you used them in real code?
- Explain prototype inheritance vs class syntax; when do you prefer composition?
- How do you handle error boundaries and runtime errors in front-end apps?
- TypeScript: how do you design types for API responses and avoid “any” creep?
- Explain generics with an example from your code (e.g., reusable table/form components).
- How do you enforce code quality (linting, formatting, type checks) in CI?

7) Performance, Security & Reliability

- How do you measure front-end performance (Lighthouse, Web Vitals, profiler)? What metrics matter most for your apps?
- Describe a performance issue you diagnosed end-to-end (symptoms → root cause → fix).
- How do you optimize bundles (tree shaking, code splitting, dependency audits)?
- What are your strategies for large tables/graphs/dashboards (virtualization, memoization, pagination)?
- How do you handle authentication/authorization in the UI (token storage, refresh, route protection)?
- Explain common web security risks (XSS, CSRF) and what you do in the front end to reduce risk.
- How do you design for observability (logging, tracing IDs, error reporting) in UI + backend integration?

8) Testing & Quality (Jasmine/Karma + overall strategy)

- What is your test pyramid for front-end applications (unit/integration/e2e)?
- How do you make unit tests stable and not brittle (selectors, fixtures, mocking)?
- How do you test API integration and error states (timeouts, 4xx/5xx, empty data)?
- How do you ensure quality during fast delivery (PR checks, review checklist, definition of done)?

9) Backend/Integration

- How do you design REST APIs to support UI needs (pagination, filtering, sorting, partial responses)?
- Tell me about a tricky integration issue between UI and backend and how you resolved it.
- How do you handle API versioning and backward compatibility for long-running enterprise apps?
- Node/Express: how do you structure middleware and error handling?
- Spring Boot: what patterns have you used for auth, controllers, and DTO mapping?

- Python services (Flask/FastAPI): how did you handle configuration, logging, and deployment readiness?

10) Cloud & Data Platform Exposure

- Azure fundamentals: which services have you used directly (App Service, storage, CI/CD)? What was your role?
- Databricks: how did the UI interact with Databricks workloads (jobs, notebooks, APIs)?
- GCP services: which services did you integrate with, and what were the key constraints (IAM, networking, quotas)?
- How do you manage environment configs (dev/qa/prod) and secrets for front-end builds?

11) Generative AI / AI-Driven Development

- You mention integrating AI-driven tools/agents to improve development velocity. What exactly did you implement (use cases, tooling, workflow)?
- How did you evaluate quality and risk (hallucinations, security, IP leaks) when implementing these AI tools?